

AWS Keys & Encryption Notes

1. Overview

AWS provides multiple ways to handle encryption and key management. Understanding *who manages the keys* and *where encryption happens* is critical.

2. Types of Encryption in AWS

2.1 Server-Side Encryption (SSE)

Encryption is handled by AWS after data is sent.

Types:

- **SSE-S3**
 - Keys managed by S3
 - Fully automatic
 - Least control
 - **SSE-KMS**
 - Uses AWS Key Management Service (KMS)
 - More control + audit logs (CloudTrail)
 - Supports key policies and IAM
 - **SSE-C (Customer-Provided Keys)**
 - You provide the key per request
 - AWS does NOT store the key
 - More operational complexity
-

2.2 Client-Side Encryption

- Data is encrypted *before* sending to AWS
- You fully control the keys
- AWS never sees unencrypted data

Tools:

- AWS SDK encryption libraries

- AWS Encryption SDK
-

3. AWS KMS (Key Management Service)

3.1 What is KMS?

A managed service to create and control encryption keys.

3.2 Key Types in KMS

1. Customer Managed Keys (CMK)

- Created and managed by you
- Full control (enable/disable, rotation)
- Can define key policies

2. AWS Managed Keys

- Created by AWS for your service (e.g., S3, EBS)
- Automatically rotated
- Limited control

3. AWS Owned Keys

- Fully managed by AWS
 - Not visible to you
 - Used internally
-

3.3 Key Types by Material

- **Symmetric Keys (Most Common)**

- Single key for encrypt + decrypt
- Used in almost all AWS services

- **Asymmetric Keys**

- Public + Private key pair
 - Used for:
 - Encryption/decryption
 - Digital signatures
-

4. Key Concepts in KMS

4.1 Envelope Encryption

- Data is encrypted using a **Data Key**
- Data Key is encrypted using a **Master Key (KMS Key)**

Flow: 1. KMS generates a data key 2. Data encrypted using data key 3. Data key encrypted with KMS key

4.2 Data Keys

- Generated using KMS (`GenerateDataKey`)
 - Used locally to encrypt large data
 - Improves performance
-

4.3 Key Policies vs IAM Policies

- **Key Policy** (Primary control)
 - Attached directly to KMS key
 - Defines who can use the key
 - **IAM Policy**
 - Grants permissions to users/roles
 - Works along with key policy
-

4.4 Grants

- Temporary permissions
 - Used by AWS services
 - Avoids modifying key policy frequently
-

5. Key Rotation

- Automatic rotation (only for symmetric CMKs)
 - Happens every 1 year
 - Keeps same key ID, changes backing key
-

6. Comparison Summary

Feature	SSE-S3	SSE-KMS	SSE-C	Client-side
Key Ownership	AWS	You (via KMS)	You	You
Control Level	Low	Medium/High	High	Full
Complexity	Low	Medium	High	High
Audit Logs	No	Yes	No	No

7. When to Use What

- Use **SSE-S3** → simple encryption, no control needed
- Use **SSE-KMS** → need auditing, control, compliance
- Use **SSE-C** → strict control, external key management
- Use **Client-side encryption** → maximum security, zero trust AWS

8. Important Exam Points (SSA-C03)

- KMS integrates with most AWS services (S3, EBS, RDS, Lambda)
- CloudTrail logs all KMS API calls
- Envelope encryption is key concept
- KMS has request limits (important for high throughput)
- Use data keys for performance optimization

9. Quick Memory Tips

- **SSE-S3 = AWS handles everything**
- **SSE-KMS = Control + Audit**
- **SSE-C = You send key every time**
- **Client-side = Encrypt before upload**

10. Real-World Example

- Banking app:
- Use **KMS + envelope encryption**
- Store encrypted data in S3
- Use IAM roles for access

- Highly sensitive system:
 - Use **client-side encryption**
 - Keys stored outside AWS (HSM)
-

End Notes

Mastering AWS keys is about understanding:

- Who controls the key
- Where encryption happens
- Level of security vs complexity